

Mini-proyecto U2: Agenda e Inventario Inteligentes

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Ana Panamito Royel Jima Daniel Saavedra Anderson Coello
Asignatura	Estructura de Datos
Ciclo	3
Unidad	2
Resultado de aprendizaje de la unidad	Diseñar e implementar un módulo que gestione citas y stock aplicando ordenación (Burbuja, Selección, Inserción) y búsqueda (secuencial —primera, última, findAll, centinela— y binaria en arreglos ordenados). Justificar con evidencias cuándo conviene cada algoritmo según datos y estructura (arreglo vs SLL).
Título de la Práctica	Agenda e Inventario Inteligentes
Nombre del Docente	NAVAS CASTELLANOS ANDRES ROBERTO
Fecha	Martes 16 de diciembre
Horario	07h:30 - 10h:30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Diseñar e implementar un módulo que gestione citas y stock aplicando ordenación (Burbuja, Selección, Inserción) y búsqueda (secuencial —primera, última, findAll, centinela— y binaria en arreglos ordenados). Justificar con evidencias cuándo conviene cada algoritmo según datos y estructura (arreglo vs SLL).

3. Materiales, Reactivos

- Guía de la práctica y requerimientos del proyecto.
- Datasets en formato **CSV** (citas, pacientes, inventario).
- Casos de prueba para validación de búsquedas y ordenación.
- Documentación teórica sobre algoritmos de búsqueda y ordenación.

4. Equipos y Herramientas

- JDK OpenJDK 23 (obligatorio).
- IDE IntelliJ IDEA Community.
- Visual Studio Code.
- Sistema de control de versiones Git con repositorio en GitHub.
- Terminal de consola con soporte para ANSI Colors.
- EVA / Moodle institucional para la entrega del proyecto.
- Herramientas de documentación: README en Markdown y editor ofimático.

5. Procedimiento / Metodología Ejecutada

Para el desarrollo de la presente práctica se siguió una metodología estructurada y progresiva, orientada a garantizar una implementación correcta, medible y comparativa de los algoritmos de búsqueda y ordenación. Las principales etapas fueron:

1. **Análisis del problema y definición del alcance**
Se identificó como objetivo diseñar un módulo que gestione citas y stock, incorporando algoritmos de ordenación (Burbuja, Selección e Inserción) y búsqueda (secuencial, centinela y binaria), junto con la recolección de estadísticas de rendimiento.
2. **Diseño de la arquitectura del sistema**
El proyecto se organizó en paquetes especializados (data, search, sorting, stats, io, presentation), aplicando separación de responsabilidades para facilitar mantenimiento y extensibilidad.
3. **Implementación de algoritmos de ordenación**
Se desarrollaron los algoritmos Burbuja, Selección e Inserción respetando su comportamiento teórico, incorporando métricas de comparaciones, intercambios y tiempo de ejecución, necesarias para su posterior análisis.
4. **Implementación de algoritmos de búsqueda**
Se implementaron búsquedas secuenciales (first, last, findAll, centinela), búsqueda binaria sobre arreglos ordenados y búsquedas sobre listas enlazadas simples (SLL), registrando estadísticas de cada ejecución.
5. **Gestión y registro de estadísticas**
Se creó un repositorio central de estadísticas que almacena los resultados de cada búsqueda y ordenación, permitiendo comparaciones consistentes entre algoritmos.
6. **Visualización de estadísticas en consola**
Se desarrollaron histogramas verticales con caracteres ASCII y colores ANSI para representar visualmente los tiempos de ejecución y facilitar la comparación directa entre algoritmos.
7. **Pruebas y validación**
Se realizaron pruebas con datasets de distintos tamaños y estados (ordenados, desordenados, duplicados), verificando la coherencia de las estadísticas y el comportamiento esperado según la complejidad teórica de cada algoritmo.
8. **Ajustes finales y documentación**
Finalmente, se refactorizó el código para asegurar consistencia en las métricas, se documentó el proyecto y se preparó para su entrega y evaluación.



6. Resultados

A. Resultados de ordenación utilizando Burbuja

Dataset	n	Algoritmo	Comparaciones	Swaps	Tiempo (ns, mediana)	Notas
citas_100	100	Burbuja	4950	2309	1039200	
citas_100_casi_ordenadas	100	Burbuja	4797	341	752700	
inventario_500_inverso	500	Burbuja	124750	124750	7181700	
pacientes_500	500	Burbuja	123930	60719	12474300	

B. Resultados de ordenación utilizando Selección

Dataset	n	Algoritmo	Comparaciones	Swaps	Tiempo (ns, mediana)	Notas
citas_100	100	Selección	4950	94	344600	
citas_100_casi_ordenadas	100	Selección	4950	5	372300	
inventario_500_inverso	500	Selección	124750	250	3433600	
pacientes_500	500	Selección	124750	486	5983200	

C. Resultados de ordenación utilizando Inserción

Dataset	n	Algoritmo	Comparaciones	Swaps	Tiempo (ns, mediana)	Notas
citas_100	100	Inserción	2403	2309	216200	
citas_100_casi_ordenadas	100	Inserción	440	341	32200	
inventario_500_inverso	500	Inserción	124750	124750	5527200	
pacientes_500	500	Inserción	61215	60719	3820800	

D. Resultados de búsqueda

Colección	Clave/Predicado	Método	Correcto (Sí/No)	Observaciones
citas_100	PAC-0003	Busqueda lineal por ID	SI	
citas_100_casi_ordenadas	apellido	findAll (todas coincidencias)	SI	
inventario_500_inverso	ITEM-0001	last	SI	
pacientes_500	PAC-0004	findAll (todas coincidencias)	SI	

7. Conclusiones

- Se evidenció que Bubble Sort solo resulta adecuado para conjuntos pequeños o casi ordenados, mientras que Selection Sort mantiene un comportamiento constante pero con menos intercambios, e Insertion Sort mostró mejor rendimiento en datos parcialmente ordenados.
- El análisis comparativo mediante estadísticas e histogramas permitió justificar de forma objetiva la elección del algoritmo según la estructura de datos y el estado inicial de la información, fortaleciendo la toma de decisiones algorítmicas.
- En las búsquedas, los métodos secuenciales son apropiados para estructuras no ordenadas o listas enlazadas, mientras que la búsqueda binaria demostró ser más eficiente únicamente cuando los datos se encuentran previamente ordenados en arreglos.